

Stone Game DP Explanation

877. Stone Game — Intuition First

This is a **game + DP** problem.

The important thing to realize is:

We don't actually need to calculate Alice's and Bob's total stones separately.

Instead, we calculate the **maximum score difference** the current player can achieve over the other player.

1. What is happening?

Suppose:

```
piles = [5, 3, 4, 5]
```

Initially:

```

[5, 3, 4, 5]
  ↑       ↑
take    take
left    right

```

Alice can take:

```

5 from left
OR
5 from right

```

After Alice takes one pile, Bob gets the next turn.

The important part is:

Both players play optimally.

So when Alice chooses a pile, she must assume Bob will make the **best possible move for himself**.

2. The key observation

Consider any subarray:

```
piles[l ... r]
```

The current player has only **two choices**:

```

Take piles[l]
OR
Take piles[r]

```

Suppose we define:

```
dp[l][r]
```

as:

Maximum score difference that the current player can achieve over the opponent using piles $l \dots r$.

For example:

```
dp[l][r] = 7
```

means:

```
current player gets 7 more stones than opponent
```

If:

```
dp[l][r] = -3
```

then the opponent gets 3 more.

3. Why score difference is powerful

Suppose Alice takes `piles[l]`.

She gets:

```
piles[l]
```

But now Bob becomes the **current player** for the remaining range:

```
[l+1 ... r]
```

And Bob can achieve:

```
dp[l+1][r]
```

more than Alice.

Therefore Alice's advantage becomes:

```
piles[l] - dp[l+1][r]
```

Similarly, if Alice takes the right pile:

```
piles[r] - dp[l][r-1]
```

Therefore:

```
dp[l][r] =
max(
    piles[l] - dp[l+1][r],
    piles[r] - dp[l][r-1]
)
```

This is the entire DP idea.

4. Why do we subtract?

This is the most important part.

Suppose:

```
[3, 4]
```

Current player takes 3.

Remaining:

```
[4]
```

Opponent takes 4.

So current player's advantage is:

```
3 - 4 = -1
```

Our formula gives:

```
dp[0][1]
= 3 - dp[1][1]
```

Since:

```
dp[1][1] = 4
```

we get:

```
3 - 4 = -1
```

Correct.

5. Base case

When there is only one pile:

```
[5]
```

The current player simply takes it.

So:

```
dp[i][i] = piles[i]
```

Diagram:

```
dp[i][i]
  [5]
   ↓
current player takes 5
difference = +5
```

6. Building the DP table

Let's use:

```
piles = [5, 3, 4, 5]
```

Index:

```
      0  1  2  3
piles = 5  3  4  5
```

Length = 1

```
dp[0][0] = 5
dp[1][1] = 3
dp[2][2] = 4
dp[3][3] = 5
```

Table:

	0	1	2	3
0	5			
1		3		
2			4	
3				5

7. Length = 2

[5,3]

Current player can take 5:

```
5 - dp[1][1]
= 5 - 3
= 2
```

Or take 3:

```
3 - dp[0][0]
= 3 - 5
= -2
```

Choose maximum:

```
dp[0][1] = max(2, -2)
          = 2
```

[3,4]

take 3:

```
3 - 4 = -1
```

take 4:

```
4 - 3 = 1
```

Therefore:

```
dp[1][2] = 1
```

[4,5]

take 4:

$$4 - 5 = -1$$

take 5:

$$5 - 4 = 1$$

Therefore:

```
dp[2][3] = 1
```

Table:

	0	1	2	3
0	5	2		
1		3	1	
2			4	1
3				5

8. Length = 3

Consider:

```
[5,3,4]
```

Current player has two choices.

Take left 5

Remaining:

```
[3,4]
```

Opponent's advantage there is:

```
dp[1][2] = 1
```

So:

$$5 - 1 = 4$$

Take right 4

Remaining:

```
[5,3]
```

Opponent's advantage:

```
dp[0][1] = 2
```

So:

$$4 - 2 = 2$$

Take maximum:

```
dp[0][2] = max(4,2)
          = 4
```

For:

```
[3,4,5]
```

Take 3:

$$3 - dp[2][3] \\ = 3 - 1$$

= 2

Take 5:

```
5 - dp[1][2]
= 5 - 1
= 4
```

Therefore:

```
dp[1][3] = 4
```

Table:

	0	1	2	3
0	5	2	4	
1		3	1	4
2			4	1
3				5

9. Finally, the whole array

Now:

```
[5,3,4,5]
```

Choice 1 — take left 5

Remaining:

```
[3,4,5]
```

Opponent can achieve:

```
dp[1][3] = 4
```

Therefore:

```
5 - 4 = 1
```

So Alice can guarantee an advantage of 1.

Choice 2 — take right 5

Remaining:

```
[5,3,4]
```

Opponent can achieve:

```
dp[0][2] = 4
```

Therefore:

```
5 - 4 = 1
```

So:

```
dp[0][3] = max(1,1)
= 1
```

Final table:

	0	1	2	3	
0	5	2	4	1	← FINAL
1		3	1	4	
2			4	1	
3				5	

Final result:

```
dp[0][3] = +1
```

Meaning:

Alice's score - Bob's score = +1

Therefore:

Alice wins

Return:

```
C++
true
```

10. C++ DP solution

```
C++
class Solution {
public:
    bool stoneGame(vector<int>& piles) {
        int n = piles.size();
        vector<vector<int>> dp(n, vector<int>(n));

        // Base case:
        // Only one pile is available
        for (int i = 0; i < n; i++) {
            dp[i][i] = piles[i];
        }

        // Build intervals of increasing length
        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                // Take left pile
                int takeLeft = piles[l] - dp[l + 1][r];

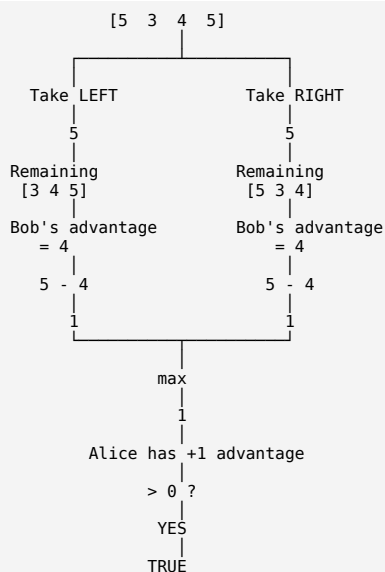
                // Take right pile
                int takeRight = piles[r] - dp[l][r - 1];

                dp[l][r] = max(takeLeft, takeRight);
            }
        }

        // Positive difference means Alice wins
        return dp[0][n - 1] > 0;
    }
};
```

11. The complete algorithm

Think of the whole process like this:



12. The most important intuition

The DP isn't really storing:

Alice's score

or:

Bob's score

It stores:

CURRENT PLAYER'S ADVANTAGE

That's why the recurrence is:

```
C++
dp[l][r] = max(
    piles[l] - dp[l+1][r],
    piles[r] - dp[l][r-1]
);
```

The pattern is:

```

CURRENT PLAYER
  |
chooses left/right
  |
takes some stones
  |
OTHER PLAYER becomes
the "current player"
  |
dp[remaining]
  |
subtract that result
  |
CURRENT PLAYER'S
  advantage
```

This is a very important **two-player game DP pattern**:

My advantage = what I take – opponent's best advantage from the remaining state.

Complexity

There are $O(n^2)$ intervals:

(l, r)

and each interval takes $O(1)$ work.

Therefore:

```
Time =  $O(n^2)$ 
Space =  $O(n^2)$ 
```

One extra observation for this specific LeetCode problem

For **LeetCode 877**, because the number of piles is even and all piles are positive, there is actually a mathematical strategy proving **Alice always wins**. So the problem can be solved simply with:

```
C++
class Solution {
public:
    bool stoneGame(vector<int>& piles) {
        return true;
    }
};
```

But the **DP solution is much more valuable for learning**, because the same $dp[l][r] = \max(\text{take} - \text{opponent})$ technique appears in many harder interval game problems.

Absolutely. For the same **Stone Game**, the memoization version is **top-down DP**.

Memoization C++

```
C++
class Solution {
```

```
public:
    int solve(vector<int>& piles, int l, int r, vector<vector<int>>& dp) {
        // Base case: only one pile
        if (l == r) {
            return piles[l];
        }

        // Already calculated
        if (dp[l][r] != -1) {
            return dp[l][r];
        }

        // Take left pile
        int takeLeft = piles[l] - solve(piles, l + 1, r, dp);

        // Take right pile
        int takeRight = piles[r] - solve(piles, l, r - 1, dp);

        // Store the best possible score difference
        return dp[l][r] = max(takeLeft, takeRight);
    }

    bool stoneGame(vector<int>& piles) {
        int n = piles.size();
        vector<vector<int>> dp(n, vector<int>(n, -1));

        int difference = solve(piles, 0, n - 1, dp);

        return difference > 0;
    }
};
```

How recursion works

For:

```
piles = [5, 3, 4, 5]
```

We start:

```
solve(0,3)
```

It has two choices:

```

solve(0,3)
├── take 5
│   └── 5 - solve(1,3)
└── take 5
    └── 5 - solve(0,2)
```

Then each smaller problem again has two choices:

```

solve(l,r)
├── take piles[l]
│   └── piles[l] - solve(l+1,r)
└── take piles[r]
    └── piles[r] - solve(l,r-1)
```

Finally:

```
C++
dp[l][r] = max(takeLeft, takeRight);
```

The important meaning of dp[l][r]

```

dp[l][r]
└── maximum score difference
    the CURRENT PLAYER can guarantee
    from piles[l...r]
```

So when we do:

```
C++
piles[l] - solve(l + 1, r)
```

it means:

```
I take piles[l]
```

```

      ↓
I gain piles[l]
      ↓
Opponent plays optimally on remaining piles
      ↓
Opponent gets advantage = solve(l+1,r)
      ↓
My final advantage =
piles[l] - opponent's advantage

```

That **subtracting the recursive result** is the key idea behind this game DP.

Yes — this is **exactly the important difference between memoization and tabulation**.

The key idea is:

The loops in tabulation are basically replacing the recursion calls of memoization.

Let's convert your code step by step.

1. Start with memoization

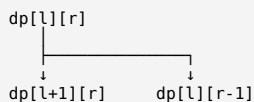
Our memoization function is:

```

C++
int solve(vector<int>& piles, int l, int r, vector<vector<int>>& dp) {
    if (l == r) {
        return piles[l];
    }
    if (dp[l][r] != -1) {
        return dp[l][r];
    }
    int takeLeft =
        piles[l] - solve(piles, l + 1, r, dp);
    int takeRight =
        piles[r] - solve(piles, l, r - 1, dp);
    return dp[l][r] = max(takeLeft, takeRight);
}

```

The important dependencies are:



This is the **most important thing**.

2. Look only at the dependency

Forget the recursion for a moment.

We have:

```

C++
dp[l][r] =
    max(
        piles[l] - dp[l+1][r],
        piles[r] - dp[l][r-1]
    );

```

Therefore:



So before calculating:

```
dp[l][r]
```

we MUST have calculated:

```

dp[l+1][r]
dp[l][r-1]

```

This dependency tells us **how to write the tabulation loops**.

3. Why memoization doesn't need loops

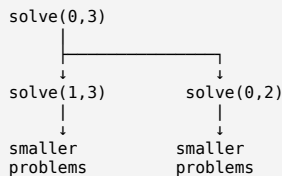
In memoization, we start from the final problem:

```
C++
solve(0, n-1)
```

For example:

```
solve(0,3)
```

The recursion automatically goes downward:



The **function calls themselves decide which state gets calculated next.**

That's why you don't need explicit loops.

4. But tabulation has no recursion

In tabulation, we remove:

```
C++
solve(...)
```

So now we have to manually decide:

"In what order should I calculate `dp[l][r]`?"

We use the dependency:

```
dp[l][r]
  |
  +-- needs
  |
  +-- dp[l+1][r]
  |
  +-- dp[l][r-1]
```

So we must calculate **smaller intervals first.**

5. What is a "smaller interval"?

Suppose:

```
[5, 3, 4, 5]
```

We have intervals:

Length 1:

```
[5]
[3]
[4]
[5]
```

Then:

Length 2:

```
[5,3]
[3,4]
[4,5]
```

Then:

Length 3:

```
[5,3,4]
[3,4,5]
```

Finally:

```
Length 4:
[5,3,4,5]
```

So the natural order is:

```
Length 1
  ↓
Length 2
  ↓
Length 3
  ↓
Length 4
```

That's why we use:

```
C++
for (int len = 2; len <= n; len++)
```

6. Where does the first loop come from?

In memoization, the recursion can reach different interval sizes.

For example:

```
solve(0,3)
```

can eventually calculate:

```
solve(0,2)
solve(1,3)
solve(0,1)
solve(1,2)
solve(2,3)
...
```

Instead of allowing recursion to discover these sizes, tabulation explicitly says:

```
C++
for (int len = 2; len <= n; len++)
```

Meaning:

```
Calculate all intervals of length 2
then all intervals of length 3
then all intervals of length 4
...
```

So the **first loop represents recursion depth / interval size**.

7. Where does the second loop come from?

Suppose:

```
len = 2
```

There are multiple intervals of length 2:

```
[5,3]
[3,4]
[4,5]
```

We need to calculate all of them.

So:

```
C++
for (int l = 0; l + len - 1 < n; l++)
```

moves the interval across the array.

For **len = 2**:

```
l = 0
[5,3]
↑ ↑
```

```
l r
```

then:

```
l = 1
[3,4]
↑ ↑
l r
```

then:

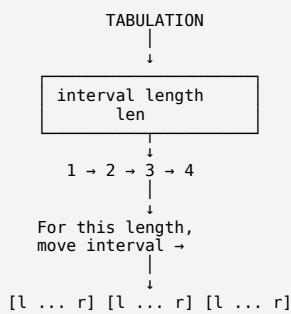
```
l = 2
[4,5]
↑ ↑
l r
```

That's the **second loop**.

8. So these two loops mean

```
C++
for (int len = 2; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;
        ...
    }
}
```

Think:



9. Let's directly convert the code

Memoization

```
C++
int solve(int l, int r) {
    if (l == r)
        return piles[l];

    int left =
        piles[l] - solve(l + 1, r);

    int right =
        piles[r] - solve(l, r - 1);

    return max(left, right);
}
```

Look at this:

```
C++
solve(l + 1, r)
solve(l, r - 1)
```

These are recursive dependencies.

Step 1 — Identify base case

Memoization:

```
C++
if (l == r)
    return piles[l];
```

Tabulation:

```
C++
for (int i = 0; i < n; i++) {
    dp[i][i] = piles[i];
}
```

Why?

Because:

```
solve(i,i)
```

becomes:

```
dp[i][i]
```

So we initialize the diagonal.

	0	1	2	3
0	5			
1		3		
2			4	
3				5

10. Step 2 — Convert recursive calls

Memoization:

```
C++
piles[l] - solve(l + 1, r)
```

becomes:

```
C++
piles[l] - dp[l + 1][r]
```

And:

```
C++
piles[r] - solve(l, r - 1)
```

becomes:

```
C++
piles[r] - dp[l][r - 1]
```

So:

```
C++
dp[l][r] = max(
    piles[l] - dp[l + 1][r],
    piles[r] - dp[l][r - 1]
);
```

11. Step 3 — Figure out the order

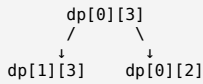
We need:

```
dp[l+1][r]
dp[l][r-1]
```

before:

```
dp[l][r]
```

For example:



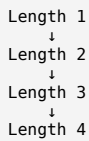
Those are intervals of length 3.

And:



Those are length 2.

Therefore:



Hence:

```

C++
for (int len = 2; len <= n; len++)

```

12. Complete conversion

Memoization

```

C++
int solve(int l, int r) {
    if (l == r)
        return piles[l];

    int takeLeft =
        piles[l] - solve(l + 1, r);

    int takeRight =
        piles[r] - solve(l, r - 1);

    return max(takeLeft, takeRight);
}

```

Tabulation

```

C++
for (int i = 0; i < n; i++) {
    dp[i][i] = piles[i];
}

for (int len = 2; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        int takeLeft =
            piles[l] - dp[l + 1][r];

        int takeRight =
            piles[r] - dp[l][r - 1];

        dp[l][r] = max(takeLeft, takeRight);
    }
}

```

13. The mental model you should remember

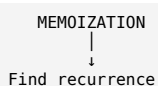
Don't memorize:

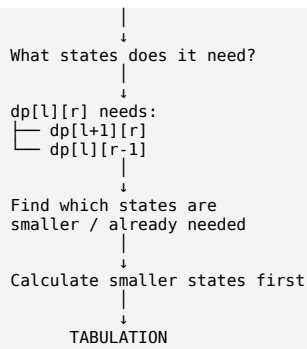
```

C++
for (len...)
for (l...)

```

Instead memorize this **conversion process**:





For this problem:

```

dp[l][r]
├ dp[l+1][r]
└ dp[l][r-1]
  
```

Both represent SMALLER intervals
 ↓
 Calculate by interval length
 ↓
 for len = 1 → n

And because there are **many intervals with the same length**:

```

len = 2
↓
[0,1]
[1,2]
[2,3]

len = 3
↓
[0,2]
[1,3]

len = 4
↓
[0,3]
  
```

That's why you see **two loops**.

In one sentence:

Memoization uses recursion to discover the dependency order automatically; tabulation removes recursion, so the loops explicitly generate states in that dependency order.

This idea is extremely important for converting **any DP memoization** → **tabulation**.
 ChatGPT can make mistakes. Check important info.